

EDF (Earliest Deadline First) scheduling

This algorithm is very intuitive and extremely easy to understand. At each scheduling point the task with the closest deadline is executed. EDF has proved itself to be the optimal single-processor scheduling algorithm for real-time systems. If a set of tasks is not schedulable by EDF, then no other scheduling algorithm will be able to feasibly schedule a task set. EDF is generally implemented using either heaps or FIFO priority queues.

RMA (Rate Monotonic Algorithm) scheduling

RMA is a static priority algorithms, by a static priority algorithm we mean that each task is given a static priority based, in this case based on it's rate or period as opposed to other cases when the priority based on deadline, which can vary. Lower the occurrence rate of a task, lower is the priority assigned to it. A task having high occurrence rate accordingly given a high priority. RMA is the most optimal static real time task scheduling algorithm. RMA is the most common real time scheduling algorithm. Nearly all the commercially available operating systems support RMA. Tasks are divided among multiple queues based on their priority levels. All tasks at a single level are then scheduled using time slicing or in a round robin manner.

Inter process communication and synchronisation

Different tasks running on an embedded OS share the same hardware and software and it's often the case that they're dependent on other tasks to provide some of the functionality for the work that they have to undertake. For these reasons embedded OSes provide various methods and mechanisms for communication and synchronisation. These mechanism are based in some combination of shared memory, message passing and signaling.

As mentioned earlier most of the advanced scheduling algorithms use preemption, i.e. a higher priority task swaps out a lower priority one at a scheduling point. This opens a new can of worms, that stem from the fact that control is taken away from a task which is not ready to give up control. For instance, if a task is writing to a shared memory location and was preempted, the next process would see corrupted data when it tried to access the same location. Or, if a task where waiting for input and was preempted, the actual signal for the task would be lost. The solution to these problems is to use constructs like mutexes and spin locks in the critical regions of the code to prevent preemption when a task is undertaking some critical operation.